

在 Cloud Run 使用 RTX 6000 Pro GPU，透過 Gemma 模型執行推論

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/cloud-run-gpu-rtx-pro-6000?hl=zh-tw>

步驟數：9

瞭解如何在 Cloud Run GPU 上使用 RTX Pro 6000 晶片執行推論作業

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 設定 Cloud Storage](#)
4. [4. 擷取模型權重](#)
5. [5. 建立服務](#)
6. [6. 部署至 Cloud Run](#)
7. [7. 測試服務](#)
8. [8. 恭喜！](#)
9. [9. 清除所用資源](#)

1. 簡介

注意：這項功能適用《[服務專屬條款](#)》中「一般服務條款」一節的《正式發布前產品條款》。正式發布前功能是依「原樣」提供，支援服務可能受限。詳情請參閱[推出階段說明](#)

總覽

課程內容

- 如何在 Cloud Run RTX 6000 Pro GPU 上部署 Gemma 模型
 - 如何在容器啟動期間從 Cloud Storage 同時下載模型
-

2. 設定和需求

注意：本程式碼研究室使用較小的 **Gemma 3 1B** 模型，因此您可以在 Cloud Shell 等儲存空間有限的環境中完成步驟。本程式碼研究室會示範如何在容器啟動期間，從 Cloud Storage 同時載入模型。詳情請參閱 [GPU 最佳做法指南](#)。如要使用較大的模型，可以在本程式碼研究室中將 `gemma3:1b` 全面換成 `gemma3:27b`。

設定本程式碼研究室全程會用到的環境變數：

```
export PROJECT_ID=<YOUR_PROJECT_ID>

export REGION=europe-west4
export SERVICE_NAME=gemma-rtx-codelab

# set the project
gcloud config set project $PROJECT_ID
```

啟用本程式碼研究室所需的 API

□□□

```
gcloud services enable artifactregistry.googleapis.com \
  cloudbuild.googleapis.com \
  run.googleapis.com \
  compute.googleapis.com
```

為 Codelab 建立資料夾

□□□

```
mkdir codelab-rtx
cd codelab-rtx
```

啟用子網路的 Private Google Access，從 Cloud Storage 載入機器學習模型時可獲得最佳效能。如要瞭解詳情，請參閱 [從 Cloud Storage 載入模型](#) 的 GPU 最佳做法文件。

□□□

```
gcloud compute networks subnets update default \
  --region=europe-west4 \
  --enable-private-ip-google-access
```

步驟 3 · 約 0 分鐘

3. 設定 Cloud Storage

首先，請建立 Cloud Storage bucket 來儲存模型權重。

建立專屬 bucket

```
# Generate a unique bucket name
export MODEL_BUCKET="${PROJECT_ID}-rtx-codelab-$(python3 -c 'import uuid;
print(str(uuid.uuid4())[:8])')"
echo "Bucket name: $MODEL_BUCKET"

# Create the regional bucket
gcloud storage buckets create gs://$MODEL_BUCKET \
  --location=$REGION \
  --uniform-bucket-level-access
```

步驟 4 · 約 0 分鐘

4. 擷取模型權重

接著，將 Gemma 3 模型下載至本機目錄，然後上傳至 Cloud Storage bucket。

安裝 Ollama

你可以執行下列指令來安裝 Ollama：

```
curl -fsSL https://ollama.com/install.sh | sh
```

下載模型

為下載的模型建立目錄。

```
mkdir model-weights
```

您將使用兩個終端機分頁執行這項程序：一個用於執行 Ollama 伺服器，另一個用於擷取模型。

終端機 1 (伺服器)：

傳遞下載模型的所在位置，啟動伺服器。這項指令會繼續執行。

```
OLLAMA_MODELS=$(pwd)/model-weights ollama serve
```

終端機 2 (用戶端)：開啟新的終端機分頁，然後下載模型。用戶端會自動與執行中的伺服器通訊。

```
# note if you wish to use a larger model, you can change this to gemma3:27b  
ollama pull gemma3:1b
```

返回終端機 1：終端機 2 下載完成後，返回終端機 1 並按下 **Ctrl+C** 停止伺服器。

上傳至 Cloud Storage

現在請將權重上傳至 bucket。`gcloud storage` 會自動處理平行上傳作業，加快上傳速度。

```
gcloud storage cp -r ./model-weights/* gs://$MODEL_BUCKET/
```

(選用) 清除本機權重

模型現在已儲存在 Cloud Storage，請移除本機副本。

```
rm -rf model-weights
```

步驟 5 · 約 0 分鐘

5. 建立服務

首先，請為服務建立資料夾。

```
mkdir rtx-service
cd rtx-service
```

建立含有以下內容的 `Dockerfile`

```
FROM ollama/ollama:latest

# Install Google Cloud CLI
RUN apt-get update && apt-get install -y curl gnupg && \
    echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] \
http://packages.cloud.google.com/apt cloud-sdk main" | tee -a /etc/apt/sources.list.d/google- \
cloud-sdk.list && \
    curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | apt-key --keyring \
/usr/share/keyrings/cloud.google.gpg add - && \
    apt-get update && apt-get install -y google-cloud-cli && \
    apt-get clean && rm -rf /var/lib/apt/lists/*

# Listen on all interfaces, port 11434
ENV OLLAMA_HOST 0.0.0.0:11434

# Store model weight files in /models
ENV OLLAMA_MODELS /models

# Reduce logging verbosity
ENV OLLAMA_DEBUG false

# Never unload model weights from the GPU
ENV OLLAMA_KEEP_ALIVE -1

# Copy and set up the startup script
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

# Start using the entrypoint script
ENTRYPOINT ["/entrypoint.sh"]
```

建立名為 `entrypoint.sh` 的檔案，並加入以下內容：

```
#!/bin/bash
set -e

# Ensure OLLAMA_MODELS directory exists
mkdir -p $OLLAMA_MODELS

# Download model weights from GCS if MODEL_BUCKET is set
if [ -n "$MODEL_BUCKET" ]; then
    echo "Downloading model weights from gs://$MODEL_BUCKET..."
    # gcloud storage handles concurrent downloads automatically
    gcloud storage cp -r "gs://$MODEL_BUCKET/*" "$OLLAMA_MODELS/"
else
    echo "MODEL_BUCKET not set. Skipping download."
fi

# Start Ollama
exec ollama serve
```

步驟 6 · 約 0 分鐘

6. 部署至 Cloud Run

在本節中，您將使用 `gcloud run deploy` 部署服務。這項指令會從來源建構容器，並部署至 Cloud Run，同時設定必要的 GPU 和網路。

建立服務帳戶

為這個應用程式建立專屬服務帳戶，並只授予必要權限。

```
# Create a dedicated service account
gcloud iam service-accounts create rtx-codelab-identity \
  --display-name="RTX Codelab Identity"

# Grant permission to read from the model bucket
gcloud storage buckets add-iam-policy-binding gs://$MODEL_BUCKET \
  --member="serviceAccount:rtx-codelab-identity@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

# Grant access to the Compute Engine network for the Cloud Run service identity
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:rtx-codelab-identity@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/compute.networkUser"

# Grant access to the Compute Engine network for the default service account
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --
format='value(projectNumber)')@serverless-robot-prod.iam.gserviceaccount.com" \
  --role="roles/compute.networkUser"
```

網路設定

如要以最佳效能下載大型模型，請使用**直接虛擬私有雲輸出**。這樣一來，容器就能透過 Google 私人網路存取 Cloud Storage，略過公用網際網路和 NAT 閘道。`gcloud run deploy` 指令會使用下列旗標：

- `--network`：連線至 `default` 虛擬私有雲 (請確認這個網路存在，且在您的區域中設有已啟用 Private Google Access 的子網路)。
- `--subnet`：您所在區域的特定子網路 (如果使用預設網路，通常為 `default`)。
- `--vpc-egress`：設為 `all-traffic`，強制所有輸出流量都必須通過虛擬私有雲。

部署指令

```
gcloud beta run deploy $SERVICE_NAME \  
  --source . \  
  --region $REGION \  
  --project $PROJECT_ID \  
  --no-allow-unauthenticated \  
  --port 11434 \  
  --service-account rtx-codelab-identity@$PROJECT_ID.iam.gserviceaccount.com \  
  --cpu 20 --memory 80Gi \  
  --gpu 1 \  
  --gpu-type nvidia-rtx-pro-6000 \  
  --set-env-vars MODEL_BUCKET=$MODEL_BUCKET \  
  --network default \  
  --subnet default \  
  --vpc-egress all-traffic \  
  --no-gpu-zonal-redundancy
```

步驟 7 · 約 0 分鐘

7. 測試服務

部署完成後，您可以使用 Ollama API 與 Gemma 3 模型互動。

取得服務網址

擷取已部署 Cloud Run 服務的網址。

```
SERVICE_URL=$(gcloud run services describe $SERVICE_NAME --region $REGION --format
'value(status.url)')
echo "Service URL: $SERVICE_URL"
```

執行推論

使用 `curl` 將提示傳送至模型。您可以設定 `"stream": false`，以單一 JSON 物件取得完整的回應，並使用 `jq` 僅擷取文字。

注意：如果您使用較大的模型 (例如 `gemma3:27b`)，請變更下方 JSON 中的模型名稱。

```
curl -s "$SERVICE_URL/api/generate" \
-H "Authorization: Bearer $(gcloud auth print-identity-token)" \
-H "Content-Type: application/json" \
-d '{
  "model": "gemma3:1b",
  "prompt": "Why is the sky blue?",
  "stream": false
}' | jq -r '.response'
```

步驟 8 · 約 0 分鐘

8. 恭喜！

恭喜您完成本程式碼研究室！

建議參閱 [Cloud Run](#) 說明文件。

涵蓋內容

- 如何在 Cloud Run RTX 6000 Pro GPU 上部署 Gemma 模型
 - 如何在容器啟動期間從 Cloud Storage 同時下載模型
-

步驟 9 · 約 0 分鐘

9. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除專案或個別資源。

選項 1：刪除資源

刪除 Cloud Run 服務

```
gcloud run services delete $SERVICE_NAME \  
  --region $REGION \  
  --quiet
```

刪除服務帳戶

```
gcloud iam service-accounts delete \  
  rtx-codelab-identity@$PROJECT_ID.iam.gserviceaccount.com \  
  --quiet
```

刪除 Cloud Storage bucket

```
gcloud storage rm --recursive gs://$MODEL_BUCKET
```

刪除容器映像檔

這個建構作業會在 Artifact Registry 中建立容器映像檔。您可以找出圖片名稱並刪除。

列出映像檔，找出確切名稱 (通常為 gcr.io/PROJECT_ID/SERVICE_NAME)

```
gcloud container images list --filter="name:$SERVICE_NAME"
```

刪除圖片 (將 IMAGE_NAME 替換為上述結果)

```
gcloud container images delete <IMAGE_NAME> --force-delete-tags
```

方法 2：刪除專案

如要刪除整個專案，請前往「管理資源」，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
